

Módulo 2: Conceptos de EF Core 10 y SQLite

Este módulo explica cómo Entity Framework Core interactúa con la base de datos SQLite en nuestra aplicación.

1. El Objeto `DbContext` y `AddDbContextPool`

El `DbContext` representa la sesión con la base de datos.

En `Program.cs`:

```
builder.Services.AddDbContextPool<AppDbContext>(options =>
options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnect
ion")));
```

 **Tip Didáctico:** `AddDbContextPool` reutiliza instancias de `DbContext` en lugar de crearlas y destruirlas en cada petición HTTP, mejorando notablemente el rendimiento bajo alta carga.

2. Consultas de Lectura Eficientes con `AsNoTracking()`

Por defecto, EF Core rastrea las entidades leídas para saber si se modifican. Sin embargo, para renderizar una lista en pantalla no necesitamos seguimiento.

```
// Sin AsNoTracking (Lento para lecturas):
var tareas = await db.Tareas.ToListAsync();

// Con AsNoTracking (Rápido y eficiente en RAM):
var tareas = await db.Tareas
    .AsNoTracking()
    .Select(t => new TareaDto(t.Id, t.Descripcion, t.EstaAcabada, t.Fecha))
    .ToListAsync(ct);
```

3. Actualizaciones y Eliminaciones Directas (`ExecuteUpdateAsync` y `ExecuteDeleteAsync`)

El enfoque tradicional (2 peticiones SQL):

1. **SELECT**: Cargar la entidad de la base de datos.
2. **UPDATE**: Modificar la propiedad y llamar a `SaveChangesAsync()`.

El enfoque moderno en EF Core 10 (1 sola petición SQL):

```
// Actualiza directamente en la base de datos sin traer los registros a memoria
await db.Tareas
    .Where(t => t.Id == id)
    .ExecuteUpdateAsync(s => s.SetProperty(t => t.EstaAcabada, t =>
!t.EstaAcabada), ct);
```

4. Migraciones y SQLite

Las migraciones traducen los cambios de código C# en scripts DDL de SQLite:

- `dotnet ef migrations add InitialCreate`: Genera el código C# que describe la creación de la tabla `Tareas`.
- `dotnet ef database update`: Aplica la migración en el archivo `app.db`.